

Exerciții — moștenire și polimorfism

Se lucrează în proiectul tău, peste `User` / `Student` / `Teacher` · Java · Adrian

Ce ai voie să folosești: clase, câmpuri `private` / `protected`, getteri și setteri, mai mulți constructori pe aceeași clasă, `extends`, `super(...)` și `super.metoda()`, `@Override`, `instanceof`, `cast (Student) u`, `List` + `ArrayList`, `for-each`, `if/else`, `String.split(",")`, `Integer.parseInt`, `LocalDate.parse`, `System.out.println`.

Formatul de text rămâne al tău: `TIP, id, email, parola, data, an, nume, numar`.

Încă NU folosim: `abstract`, `interface`, `toString()`, `equals` / `hashCode`, `static`, citire/scriere în fișier, `try/catch`, `stream-uri`.

Partea 1 — la fel ca la clasă

1. A treia subclasă: `Admin`

ușor

Scrie `Admin extends User`, în același pachet `app.users`, exact pe tiparul lui `Teacher`:

- câmpuri private: `anAngajare (int)`, `nume (String)`, `nrConturi (int)`;
- getteri și setteri pentru toate trei;
- un constructor care primește tot textul dintr-o singură bucată și cheamă `super(text)`;
- `descriere()` suprascrisă, care începe cu `"ADMIN"` și folosește `super.descriere()`.

```
Admin a1 = new Admin("ADMIN,010,admin@scoala.com,adm123,2026-09-10,2019,Radu,42");
System.out.println(a1.descriere());
```

```
ADMIN,010,admin@scoala.com,adm123,2026-09-10,2019,Radu,42
```

Dacă îți dă „there is no default constructor available”, uită-te pe ce linie din constructor e `super(...)`.

2. O listă, trei tipuri, zero `if`

ușor

În `App`, fă o `List<User>` cu 2 studenți, 2 profesori și 1 admin. Parcurge-o cu `for-each` și afișează `u.descriere()`.

Regula exercițiului: în buclă n-ai voie să scrii niciun `if` și niciun `cast`.

```
STUDENT,001,ana@ex.com,p1,2026-09-01,2024,Ionescu,20
STUDENT,002,dan@ex.com,p2,2026-09-01,2024,Popescu,18
TEACHER,007,prof@ex.com,p3,2026-09-03,2015,Dumitru,5000
TEACHER,008,ionel@ex.com,p4,2026-09-04,2018,Ionescu,5200
ADMIN,010,admin@scoala.com,adm123,2026-09-10,2019,Radu,42
```

Referința e `User` pentru toate cinci, dar rulează versiunea fiecărui obiect. Asta e late binding, văzut cu ochiul liber.

3. Când chiar ai nevoie de `instanceof`

ușor

Peste aceeași listă, afișează pentru fiecare user o linie cu numele și **cifra lui specifică**: creditele pentru student, salariul pentru profesor, numărul de conturi pentru admin. Metodele astea nu există în `User`, deci îți trebuie `instanceof` + cast.

```
Ionescu are 20 credite
Popescu are 18 credite
Dumitru are salariul 5000
Ionescu are salariul 5200
Radu administreaza 42 conturi
```

Întâi `instanceof`, abia apoi cast — în ordinea asta, niciodată invers.

Partea 2 — un pic mai grele

4. O metodă din superclasă care cheamă o metodă suprascrisă

mai greu

Adaugă în `User` o metodă `public String tip()` care returnează `"USER"`. Suprascrie-o în `Student` (`"STUDENT"`), `Teacher` (`"TEACHER"`) și `Admin` (`"ADMIN"`).

Apoi adaugă în `User`, fără s-o suprascrii nicăieri:

```
public String eticheta() {
    return "[" + tip() + "] " + getEmail();
}
```

Înainte să rulezi, scrie pe hârtie ce crezi că afișează `new Student(...).eticheta()`. Apoi rulează și explică-ți diferența (sau potrivirea).

```
[STUDENT] ana@ex.com
```

`eticheta()` e scrisă în `User` și totuși nu apare `[USER]`. Cine alege versiunea lui `tip()` — codul unde e scris apelul, sau obiectul pe care se cheamă?

5. Supraîncărcare lângă suprascriere

mai greu

În `Admin` scrie **două** metode cu același nume:

```
public String raport(String titlu)
public String raport(String titlu, int an)
```

Prima returnează `titlu + ": " + nume`, a doua adaugă și anul. Cheamă-le pe rând.

Apoi răspunde în scris, în două rânduri: pentru `descriere()` cine decide ce cod rulează și **când**, iar pentru `raport(...)` cine decide și **când**. Nu e același răspuns.

Uită-te și la cei doi constructori ai lui `User` — sunt exact același mecanism ca `raport`.

6. Căutare în UserRepository

mai greu

În `UserRepository`, peste lista pe care o ai deja, adaugă:

- `public User cautuDupaId(String id)` — întoarce user-ul cu id-ul cerut, sau `null` dacă nu există;
- `public int numaraStudenti()` — câți `Student` sunt în listă.

În `App`: caută un id de student și afișează-i creditele; apoi caută un id care nu există și tratează cazul fără să crape programul.

```
Utilizator 001: 20 credite
Utilizator 555: nu exista
```

`cautaDupaId` întoarce `User`, deci n-are cum să știe de credite. Două verificări diferite îți trebuie aici: una pentru `null`, una pentru tip.

Partea 3 — provocări

7. Câmpul care nu se lasă suprascris

provocare

Adaugă `protected String rol = "USER";` în `User` și, în `Student`, o linie identică dar cu `"STUDENT"`. Apoi rulează:

```
Student s = new Student("STUDENT,001,ana@ex.com,p1,2026-09-01,2024,Ionescu,20");
User u = s;

System.out.println(s.rol);
System.out.println(u.rol);
System.out.println(((Student) u).rol);
```

Prezice cele trei linii înainte de a rula. Apoi explică într-o frază de ce `u.descriere()` se comportă altfel decât `u.rol`, deși `u` și `s` arată spre **același obiect**.

Câte câmpuri `rol` există în obiectul ăla, unul sau două?

8. Cast-ul care compilează și tot crapă

provocare

Scrie intenționat codul care greșește și rulează-l ca să vezi excepția:

```
User u = new Teacher("TEACHER,007,prof@ex.com,p3,2026-09-03,2015,Dumitru,5000");
Student s = (Student) u;
System.out.println(s.getNrCredite());
```

Notează pe fișă mesajul exact primit. Apoi răspunde: de ce compilatorul a lăsat linia 2 să treacă, deși e evident greșită când citești codul? Și de ce `String x = (String) u;` **nu** ar fi trecut de compilare?

La final, rescrie bucata astfel încât programul să afișeze `"Nu e student"` în loc să crape.

Compilatorul se uită la **tipul referinței** și la arborele de moștenire. Mașina virtuală se uită la obiectul real. Unul dintre ei are mai multă informație — care?